Section 7.3: API Lifecycle Tiers & Telemetry Management

> ***Study Set:*** *3 files total* | ***Section:*** *7.**
> ***Focus:*** *API gateways, rate limiting algorithms, resilience patterns, and distributed system protection for system design interviews.*

Executive Overview

API gateways and telemetry systems form the **control plane** of modern architectures. This section covers the mechanics of rate limiting, circuit breaking, backpressure, and graceful degradation — patterns that distinguish senior engineers from mid-level in system design interviews. Interviewers probe *why* these mechanisms exist and *how* they fail under load.

---

1. Ingress Orchestration: API Gateways

1.1 What an API Gateway Does

An API gateway is the single entry point for all client traffic. It offloads cross-cutting concerns from individual services.

```
Client Request
      ↓
API Gateway
  ├─ Authentication / Authorization (JWT, OAuth2, mTLS)
  ├─ Rate Limiting (per user, per key, global)
  ├─ Request Transformation (REST→gRPC, XML→JSON, header injection)
  ├─ Routing (path-based, header-based, canary splits)
  ├─ Observability (request logging, tracing, metrics)
  └─ Response Composition (fan-out to multiple services, aggregate)
      ↓
Target Microservices
```

Key gateway functions:

Function	Mechanism
JWT validation	Verify signature locally (no network hop) using public key
OAuth2 introspection	Call auth server to validate opaque token (adds latency)
API key lookup	Redis or in-memory cache lookup
mTLS	Client certificate validation at TLS layer
Protocol translation	REST→gRPC adapter; XML→JSON transformation
Request composition	Aggregate 3 service calls into 1 client response

Gateway vs. service mesh: - **Gateway:** North-south traffic (external clients → internal services) - **Service mesh (Envoy/Istio):** East-west traffic (service → service), mTLS, retries, circuit breaking at the infrastructure layer

Many architectures use both: gateway for external auth/rate limiting, service mesh for internal resilience.

1.2 API Versioning Strategies — Comparison

Versioning is a recurring interview question because there's no universally right answer — context determines the choice.

Strategy	Example	When to Use	Drawback
URI versioning	`GET /v1/users/123`	Public APIs, third-party integrations	URL "pollution"; clients must update
Header versioning	`Accept: application/vnd.api.v2+json`	Internal APIs, clean URL requirements	Harder to test; some proxies strip custom headers
Query param	`GET /users/123?version=2`	Rapid prototyping, backward-compat testing	Not RESTful; cache pollution
GraphQL versioning	Additive schema evolution	GraphQL APIs	Deprecation must be explicit in schema

Best practice for public APIs: URI versioning + maintain v(N-1) for at least 12 months. Communicate deprecation via `Sunset` and `Deprecation` HTTP headers.

---

2. Rate Limiting — Algorithms and Trade-offs

Rate limiting is one of the most common "how would you implement X" system design interview questions. Know the algorithms, their trade-offs, and the failure modes.

2.1 Token Bucket

```
Bucket capacity: 100 tokens
Refill rate:     10 tokens/second

Request arrives:
  IF bucket has tokens → consume 1, allow request
  IF bucket empty      → reject (429) or queue
```

Properties: - Allows bursting up to bucket capacity - Smooth steady-state rate enforcement - Simplest to implement; used by AWS API Gateway, Stripe

Numeric example:

```
t=0s:  bucket=100. User sends 100 requests → bucket=0
t=0.5s: bucket=5  (5 tokens refilled at 10/s). User sends 10 → 5 allowed, 5
rejected
t=1s:  bucket=10 (10 tokens refilled). Steady state: 10 req/s sustainable
```

2.2 Sliding Window Log

```
Window: 60 seconds, limit: 1000 requests
On each request:
  1. Remove entries older than now-60s from sorted set
  2. Count remaining entries
  3. If count < 1000 → allow, add timestamp
  4. If count ≥ 1000 → reject
```

Properties: - Exact — no boundary spikes - Memory cost: O(limit) per user (stores every timestamp) - Not suitable for millions of users without careful memory management

---

2.3 Fixed Window Counter

```
Window: 60 seconds, limit: 1000
Counter resets at each window boundary (00:00, 01:00, 02:00...)
```

Boundary spike problem (proof by example):

```
Limit: 1000 req/min, window resets at :00

t=00:59: User sends 1000 requests → all allowed (window 1 full)
t=01:00: Window resets
t=01:01: User sends 1000 requests → all allowed (window 2 fresh)

Effective rate at boundary: 2000 requests in 2 seconds
→ 60× the intended limit in a short burst
```

This is why fixed window is generally avoided for strict rate limiting.

2.4 Sliding Window Counter (Hybrid — Recommended)

Approximates sliding window log at a fraction of the memory cost.

```
def sliding_window_check(client_id, limit=1000, window=60):
    now = int(time.time())
    current_slot  = now // window
    prev_slot     = current_slot - 1
    elapsed_ratio  = (now % window) / window      # How far into current window

    current  = int(redis.get(f"{client_id}:{current_slot}") or 0)
    previous = int(redis.get(f"{client_id}:{prev_slot}")    or 0)

    # Weighted estimate: previous window's requests taper off as current fills
    effective = current + previous * (1 - elapsed_ratio)

    if effective < limit:
        redis.incr(f"{client_id}:{current_slot}")
        redis.expire(f"{client_id}:{current_slot}", window * 2)
        return True   # Allowed
    return False      # Rate limited
```

Numeric example:

```
Window = 60s, limit = 100 req/min
At t=45s (75% through window):
  previous window: 80 requests
  current window:  60 requests
```

```
  elapsed_ratio:   0.75

effective = 60 + 80 * (1 - 0.75) = 60 + 20 = 80 → allowed
```

Trade-off: ~1–2% error vs. true sliding log; memory = O(1) per user (just 2 counters).

2.5 Redis Lua Implementation (Atomic)

Race condition without atomicity: check-then-increment is a TOCTOU bug. Fix with Lua (runs atomically in Redis):

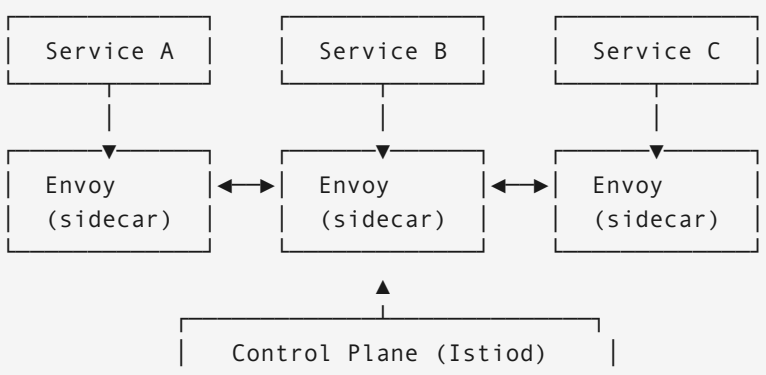
```
local key     = KEYS[1]
local limit   = tonumber(ARGV[1])
local window  = tonumber(ARGV[2])

local current = redis.call("GET", key)
if current and tonumber(current) >= limit then
    return 0  -- Rate limited
end

redis.call("INCRBY", key, 1)
if not current or tonumber(current) == 0 then
    redis.call("EXPIRE", key, window)
end
return 1  -- Allowed
```

The entire script is atomic — no concurrent increment race.

---

3. Microservice Resiliency Patterns

3.1 Service Mesh with Sidecar Proxy



```
              │  Config / certs / routing    │
              └──────────────────────────────┘
```

Envoy handles: retries, circuit breaking, mTLS, distributed tracing, traffic splitting — *without application code changes*.

Retry configuration:

```
retries:
  attempts: 3
  perTryTimeout: 2s
  retryOn: 5xx,gateway-error,reset,connect-failure
  retryBackoff:
    baseInterval: 100ms
    maxInterval: 1000ms
```

Why exponential backoff + jitter matters:

```
Without jitter (synchronized retries):
  t=0ms:   1000 requests fail
  t=100ms: 1000 simultaneous retries → thundering herd
  t=200ms: 1000 simultaneous retries → service never recovers

With jitter:
  retry_delay = random_between(0, min(cap, base * 2^attempt))
  Retries spread across time window → service can recover
```

3.2 Circuit Breaker

The circuit breaker prevents cascading failures by stopping calls to a degraded service.

State machine:

```
CLOSED  ────(5 consecutive failures)────►  OPEN
  ▲                                          │
  │                                    (30s timeout)
  │                                          ▼
  └─────(probe succeeds)──────────────  HALF-OPEN
                                     (1 probe request allowed)
```

State definitions: - **CLOSED:** Normal operation. All requests pass through. - **OPEN:** Service is failing. All requests fail-fast (no network call). Returns cached response or error immediately. - **HALF-OPEN:** Trial state. One request allowed through. Success → CLOSED. Failure → OPEN (reset timer).

Implementation:

```
class CircuitBreaker:
    def __init__(self, failure_threshold=5, timeout=30):
        self.failure_count = 0
        self.state = "CLOSED"
        self.last_failure_time = None
        self.threshold = failure_threshold
        self.timeout = timeout

    async def call(self, func, *args):
        if self.state == "OPEN":
            if time.time() - self.last_failure_time > self.timeout:
                self.state = "HALF_OPEN"
            else:
                raise CircuitOpenError("Circuit is open")

        try:
            result = await func(*args)
            if self.state == "HALF_OPEN":
                self.state = "CLOSED"
                self.failure_count = 0
            return result
        except Exception as e:
            self.failure_count += 1
            self.last_failure_time = time.time()
            if self.failure_count >= self.threshold:
                self.state = "OPEN"
            raise e
```

3.3 Dead Letter Queue (DLQ) — Poison Message Handling

```
Normal flow:
Message → Consumer → success → ack → removed

Failure flow:
Message → Consumer → fail (attempt 1)
        → Consumer → fail (attempt 2)
        → Consumer → fail (attempt 3) → DLQ + alert
```

DLQ processing: - On-call engineer inspects DLQ — determine if message is malformed (poison) or transient failure - Malformed: fix upstream producer schema; discard - Transient: replay DLQ messages after service recovery

Always set a max delivery count on queues — without it, a poison message causes an infinite retry loop that starves the consumer.

3.4 Backpressure

Backpressure is a signal from a slow consumer to upstream producers to slow down.

Reactive Streams pattern:

```
Consumer processes at 10K/s
Producer sends at 50K/s

Without backpressure: Consumer buffer fills → OOM crash

With backpressure:
  Consumer signals: "I have space for 100 more"
  Producer sends only 100
  Consumer processes, signals again
```

Practical implementations:

Mechanism	How it works
Kafka consumer lag	Monitor lag; alert/scale when lag grows
HTTP 429 (Too Many Requests)	Gateway returns 429; client backs off
TCP flow control	Kernel-level; receiver advertises window size
Queue depth	SQS/RabbitMQ depth metric triggers auto-scaling

---

4. Distributed Rate Limiting — The 1M RPS Problem

This is the canonical interview question for senior+ roles. It combines Redis, fallback design, and failure mode reasoning.

Architecture for 1M RPS across a gateway cluster:

Step 1 — Local rate limiting (first line of defense, no network hop):

```
class HybridRateLimiter:
    def __init__(self):
        # Per-node local limiter (no shared state, ~microsecond latency)
        self.local = LocalTokenBucket(capacity=1000, refill=100)
        self.redis  = RedisCluster(...)

    def check(self, client_id):
        # Fast path: if local bucket exhausted, reject immediately
```

```
        if not self.local.check(client_id):
            return False

        # Slow path: distributed check for exact enforcement
        return self.redis_sliding_window(client_id)
```

Step 2 — Redis Cluster with hash tags for consistent sharding:

```
Key format: "{client_id}:rate:{minute_bucket}"
Hash tag {client_id} ensures all keys for a client land on the same shard
→ No cross-shard coordination needed
```

Step 3 — Graceful degradation when Redis is unavailable:

```
def check_rate_limit(client_id):
    try:
        return redis_check(client_id)          # Normal path
    except RedisConnectionError:
        # Degraded mode: local-only limiting
        # May over-allow by (N_nodes × local_limit) temporarily
        logger.warning("Redis down: falling back to local rate limiting")
        return local_check(client_id)
    except Exception as e:
        # Unknown error: fail open (availability > correctness)
        logger.error(f"Rate limit error: {e}")
        metrics.increment("rate_limit_errors")
        return True  # Allow — better to over-allow than block all traffic
```

Step 4 — Circuit breaker around Redis:

```
redis_cb = CircuitBreaker(failure_threshold=10, timeout=30)

def redis_check(client_id):
    return redis_cb.call(redis.execute_script, client_id, limit, window)
```

Numeric trade-off analysis:

Scenario	Over-allocation risk	Availability
Redis UP, N=10 nodes	0%	100%
Redis DOWN, local fallback	Up to 10× limit	100%
Redis DOWN, fail closed	0%	0% (all requests blocked)

Key insight: For rate limiting (a soft correctness requirement), fail-open is almost always correct. Blocking all legitimate traffic during a Redis outage is worse than temporarily over-allocating by 10×. Reserve fail-closed for payment idempotency and auth — where the cost of an error is catastrophic.

---

5. Interview Design Problems

Problem 1 — Design Rate Limiting for a Public API Gateway

Scenario: Public API, 500K developers, each with an API key. Free tier: 100 req/min. Pro tier: 10K req/min. Enterprise: 1M req/min. How do you implement this at scale?

Solution:

Tier-aware rate limiting:

```
TIER_LIMITS = {
    'free':       {'rate': 100,       'burst': 200},
    'pro':        {'rate': 10_000,    'burst': 15_000},
    'enterprise': {'rate': 1_000_000, 'burst': 1_200_000},
}

def check(api_key):
    tier = api_key_cache.get(api_key)['tier']
    limits = TIER_LIMITS[tier]
    return sliding_window_check(api_key, limits['rate'], limits['burst'])
```

Response headers (expose limits to developers):

```
X-RateLimit-Limit:     10000
X-RateLimit-Remaining: 9847
X-RateLimit-Reset:     1718000000
Retry-After:           12        (only on 429)
```

Shard Redis by tier: Enterprise customers get dedicated Redis shards — their volume shouldn't affect free tier latency.

Problem 2 — Design a Circuit Breaker for a Payment Dependency

Scenario: Your checkout service calls a payment processor. The payment processor goes down for 2 minutes every night during maintenance. During that window, all checkout requests hang for 30 seconds (timeout) before failing, cascading to a queue buildup. Fix it.

Solution:

Circuit breaker with fallback:

```
payment_cb = CircuitBreaker(
    failure_threshold=5,
    timeout=60,              # 1 minute before probing
    half_open_max_calls=1
)

def process_payment(order):
    try:
        return payment_cb.call(payment_processor.charge, order)
    except CircuitOpenError:
        # Fail fast — no 30s hang
        # Return "pending" state, queue for retry
        queue_for_retry(order)
        return {"status": "pending", "retry_at": time.time() + 90}
```

Result: During the 2-minute outage, requests fail in <1ms instead of hanging 30 seconds. The queue buildup disappears. Recovery is automatic when the circuit half-opens and a probe succeeds.

Problem 3 — Observability: Detecting a Misbehaving Client

Scenario: Your API is seeing 5× normal error rates. You have access to gateway logs and metrics. How do you find the culprit?

Solution — Telemetry-driven triage:

```
Step 1: Error rate by endpoint
  → 95% of errors on POST /orders → narrow to that service

Step 2: Error rate by client (API key)
  → 99% of errors from api_key=abc123 → single misbehaving client

Step 3: Error type breakdown
  → 400 Bad Request (malformed payload) → client bug, not server bug

Step 4: Trace inspection (distributed tracing)
  → Trace shows payload missing required field `currency`

Step 5: Action
  → Rate limit api_key=abc123 aggressively
  → Return descriptive 400 with schema validation error
  → Contact developer with payload example
```

What metrics to always instrument at the gateway: - Request rate by `(endpoint, client_id, status_code)` - P50/P95/P99 latency by `(endpoint, upstream_service)` - Error rate with error type breakdown (4xx client vs 5xx server) - Rate limit hit rate by tier and client

---

6. Key Concepts Cheat Sheet

Concept	One-line definition
Token bucket	Fixed-capacity bucket; tokens added at constant rate; allows controlled bursting
Sliding window counter	Weighted blend of current + previous window counters; O(1) memory
Circuit breaker	Stops calls to failing services; allows recovery without thundering herd
Thundering herd	Simultaneous retries overwhelming a recovering service
Exponential backoff + jitter	Retry delay grows with each attempt, randomized to spread load
Dead Letter Queue	Messages that fail N times are routed here for manual inspection
Backpressure	Consumer signals to producer to slow down; prevents OOM
Fail-open	On error, allow traffic through (prioritize availability)
Fail-closed	On error, block traffic (prioritize correctness)
Service mesh	Infrastructure layer (sidecars) handling retries, mTLS, tracing between services
North-south traffic	External client → internal services (API gateway domain)
East-west traffic	Service → service (service mesh domain)

End of Section 7. | Return to Section 7.1 — Enterprise Integration Styles*